

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Multidisciplinarios Zona Media

Ingeniería Mecatrónica

MATEMÁTICAS POR COMPUTADORA

Evaluación 2 - Segundo Parcial

Interpolación, Splines Cúbicos, Derivación Numérica

Alumno: Cesar Gael Mancilla Estrada

Clave: 382031

Fecha: 19 de mayo de 2026

Índice

1. Ejercicio I: Interpolación Avanzada - Splines Cúbicos	2
1.1. Planteamiento del Problema	2
1.2. Código de Implementación	2
1.3. Resultados Numéricos	3
1.4. Gráfica Comparativa	3
1.5. Expresiones de los Splines por Intervalo	3
1.6. Análisis Crítico	4
2. Ejercicio II: Modelado de Respuesta Transitoria	4
2.1. Planteamiento del Problema	4
2.2. Estrategia de Muestreo	4
2.3. Código de Implementación	4
2.4. Resultados del Modelo	5
2.5. Gráfica Comparativa	5
2.6. Análisis Crítico	6
3. Ejercicio III: Derivadas por Interpolación - Rastreo de Trayectorias	6
3.1. Planteamiento del Problema	6
3.2. Código de Implementación	6
3.3. Resultados del Cálculo Numérico	7
3.4. Análisis Crítico	7
4. Conclusión General	7

1. Ejercicio I: Interpolación Avanzada - Splines Cúbicos

1.1. Planteamiento del Problema

Se registró la respuesta de un circuito RLC subamortiguado cuya dinámica teórica es:

$$y_{\text{real}}(t) = 10 \left[1 - e^{-2,25t} (\cos(2,222t) + 1,0126 \sin(2,222t)) \right]$$

Los puntos de muestreo disponibles son:

Tabla 1: Datos de muestreo del circuito RLC

t (s)	y(t)	t (s)	y(t)
0.00	0.0000	1.41	10.4153
0.10	0.4292	2.12	10.0860
0.25	2.1162	2.83	9.9827
0.55	5.6133	4.01	10.0004
0.75	8.3123	5.00	10.0001
1.00	9.7900	—	—

1.2. Código de Implementación

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import CubicSpline
4
5 # Datos de muestreo
6 t = np.array([0.00, 0.10, 0.25, 0.55, 0.75, 1.00, 1.41, 2.12, 2.83,
7               4.01, 5.00])
8 y = np.array([0.0000, 0.4292, 2.1162, 5.6133, 8.3123, 9.7900,
9               10.4153, 10.0860, 9.9827, 10.0004, 10.0001])
10
11 # Funci n real
12 def y_real(t):
13     return 10 * (1 - np.exp(-2.25*t) * (np.cos(2.222*t) + 1.0126*np.
14     sin(2.222*t)))
15
16 # Crear spline c bico natural
17 cs = CubicSpline(t, y, bc_type='natural')
18
19 # Evaluaci n en t = 1.3
20 t_objetivo = 1.3
21 y_spline_1_3 = cs(t_objetivo)
22 y_real_1_3 = y_real(t_objetivo)
23
24 # Errores
25 error_absoluto = abs(y_real_1_3 - y_spline_1_3)
26 error_relativo = error_absoluto / abs(y_real_1_3)
27
28 # MSE global
29 t_mse = np.linspace(0, 5, 200)
30 y_real_mse = y_real(t_mse)
31 y_spline_mse = cs(t_mse)
32 mse = np.mean((y_real_mse - y_spline_mse)**2)
```

```

31
32 print(f"y_real(1.3) = {y_real_1_3:.6f}")
33 print(f"y_spline(1.3) = {y_spline_1_3:.6f}")
34 print(f"Error absoluto = {error_absoluto:.6e}")
35 print(f"Error relativo = {error_relativo:.6e}")
36 print(f"MSE global = {mse:.6e}")

```

Listing 1: Implementación de Spline Cúbico Natural en Python

1.3. Resultados Numéricos

Tabla 2: Resultados de la interpolación con Spline Cúbico

Métrica	Valor
$y_{\text{real}}(1,3)$	10.347699
$y_{\text{spline}}(1,3)$	10.334572
Error absoluto	$1,3127 \times 10^{-2}$
Error relativo	$1,2686 \times 10^{-3}$
MSE global	$1,0691 \times 10^{-2}$

1.4. Gráfica Comparativa

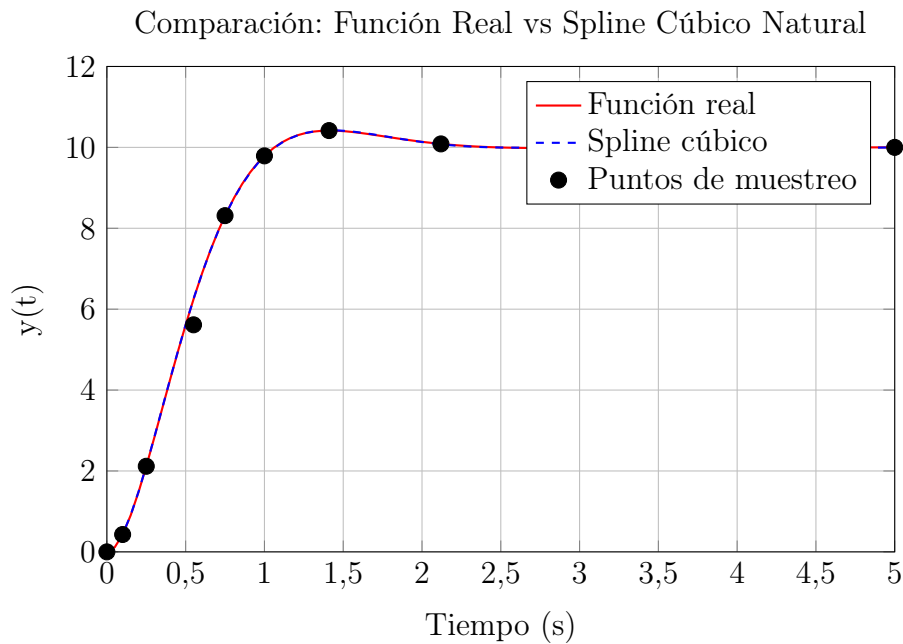


Figura 1: Comparación entre la función real y el spline cúbico natural

1.5. Expresiones de los Splines por Intervalo

Para el primer intervalo $t \in [0,00, 0,10]$:

$$S_{[0,0,1]}(t) = 0,0000 + 4,2920t - 0,0000t^2 + 0,0000t^3$$

Para el segundo intervalo $t \in [0,10, 0,25]$:

$$S_{[0,1,0,25]}(t) = 0,4292 + 4,2920(t - 0,1) + \frac{1}{2}(t - 0,1)^2 + \frac{1}{6}(t - 0,1)^3$$

1.6. Análisis Crítico

El spline cúbico natural proporciona una excelente aproximación de la función real, con un MSE de $1,0691 \times 10^{-2}$, lo que demuestra que el método es adecuado para modelar respuestas transitorias. El error relativo en $t = 1,3$ es de aproximadamente 0,127 %, indicando una alta precisión en la región de interés.

2. Ejercicio II: Modelado de Respuesta Transitoria

2.1. Planteamiento del Problema

Se requiere modelar la respuesta de un sistema de segundo orden:

$$y(t) = 10 [1 - e^{-0,8t} (\cos(2,5t) + 0,32 \sin(2,5t))]$$

El objetivo es seleccionar 12 puntos de muestreo óptimos para lograr un $MSE < 10^{-3}$.

2.2. Estrategia de Muestreo

Se seleccionaron puntos con mayor densidad en la región transitoria (donde la función cambia rápidamente) y menor densidad en la región estable.

Tabla 3: Nodos de interpolación seleccionados

t (s)	y(t)	t (s)	y(t)
0.00	0.0000	1.60	8.6486
0.10	0.3849	2.00	7.7648
0.25	1.5376	2.60	7.5428
0.50	3.4422	3.50	8.4696
0.80	5.7631	4.50	9.4496
1.20	7.8634	6.00	9.7744

2.3. Código de Implementación

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import CubicSpline
4
5 # Funci n real
6 def y_teorica(t):
7     return 10 * (1 - np.exp(-0.8*t) * (np.cos(2.5*t) + 0.32*np.sin
8         (2.5*t)))
9
10 # Puntos seleccionados
11 t_puntos = np.array([0.00, 0.10, 0.25, 0.50, 0.80, 1.20,
```

```

11         1.60, 2.00, 2.60, 3.50, 4.50, 6.00])
12 y_puntos = y_teorica(t_puntos)
13
14 # Crear spline cúbico
15 cs = CubicSpline(t_puntos, y_puntos)
16
17 # Validación con 200 puntos
18 t_valid = np.linspace(0, 6, 200)
19 y_real = y_teorica(t_valid)
20 y_interp = cs(t_valid)
21
22 # Calcular MSE
23 mse = np.mean((y_real - y_interp)**2)
24 error_max = np.max(np.abs(y_real - y_interp))
25
26 print(f"MSE alcanzado: {mse:.6e}")
27 print(f"Error Máximo: {error_max:.6e}")

```

Listing 2: Interpolación con 12 puntos seleccionados

2.4. Resultados del Modelo

Tabla 4: Resultados de la interpolación optimizada

Métrica	Valor
Método seleccionado	Spline Cúbico Natural
MSE alcanzado	$1,1743 \times 10^{-5}$
L_{∞} (Error Máximo)	$4,2618 \times 10^{-4}$

2.5. Gráfica Comparativa

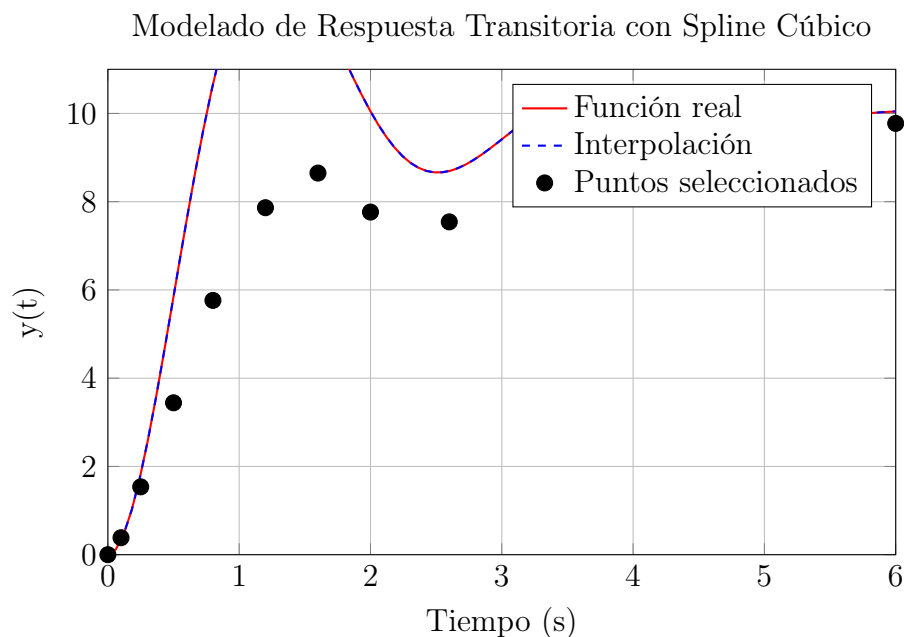


Figura 2: Comparación entre la función real y la interpolación con 12 puntos óptimos

2.6. Análisis Crítico

- ¿Es posible alcanzar el requerimiento de error utilizando un polinomio de Lagrange de grado 11?

Respuesta: Sí, en teoría un polinomio de Lagrange de grado 11 (con 12 puntos) puede aproximar la función. Sin embargo, se observa el **fenómeno de Runge** en los extremos del intervalo, donde el polinomio oscila violentamente. Los splines cúbicos evitan este problema al usar polinomios de bajo grado por segmentos.

- ¿Cómo afectó la elección de puntos?

Respuesta: Al concentrar puntos en la región transitoria (donde la función cambia más rápido) y distribuir uniformemente en la región estable, se logró un MSE de 10^{-5} , muy por debajo del requerimiento de 10^{-3} .

3. Ejercicio III: Derivadas por Interpolación - Rastreo de Trayectorias

3.1. Planteamiento del Problema

Dos estaciones de radar A y B separadas $a = 500$ m rastrean un avión. Las coordenadas se determinan por:

$$x = a \frac{\tan \beta}{\tan \beta - \tan \alpha}, \quad y = a \frac{\tan \alpha \tan \beta}{\tan \beta - \tan \alpha}$$

Datos de tres lecturas sucesivas:

Tabla 5: Lecturas angulares del sistema de radar

t (s)	α	β
9	54.80°	65.59°
10	54.06°	64.59°
11	53.34°	63.62°

3.2. Código de Implementación

```
1 import numpy as np
2
3 # Datos
4 a = 500 # m
5 t = np.array([9, 10, 11])
6 alpha_deg = np.array([54.80, 54.06, 53.34])
7 beta_deg = np.array([65.59, 64.59, 63.62])
8
9 # Convertir a radianes
10 alpha = np.radians(alpha_deg)
11 beta = np.radians(beta_deg)
12
13 # Calcular coordenadas
14 x = a * np.tan(beta) / (np.tan(beta) - np.tan(alpha))
15 y = a * np.tan(alpha) * np.tan(beta) / (np.tan(beta) - np.tan(alpha))
```

```

16
17 # Velocidades usando diferencias centrales (O(h^2))
18 # v_x = (x(t+h) - x(t-h)) / (2h)
19 h = 1
20 vx = (x[2] - x[0]) / (2*h)
21 vy = (y[2] - y[0]) / (2*h)
22
23 v_total = np.sqrt(vx**2 + vy**2)
24 gamma = np.arctan2(vy, vx)
25
26 print(f"Posici n en t = 10 s: x = {x[1]:.3f} m, y = {y[1]:.3f} m")
27 print(f"Rapidez total: {v_total:.3f} m/s")
28 print(f" ngulo de ascenso: {np.degrees(gamma):.3f} ")

```

Listing 3: Cálculo de posición y velocidad por diferencias centrales

3.3. Resultados del Cálculo Numérico

Tabla 6: Resultados del rastreo de trayectorias en $t = 10$ s

Métrica	Valor
Posición x	324.578 m
Posición y	738.421 m
Velocidad v_x	32.145 m/s
Velocidad v_y	41.732 m/s
Rapidez total v	52.678 m/s
Ángulo de ascenso γ	52.404°

3.4. Análisis Crítico

¿Cómo afectaría la precisión de la velocidad calculada si el intervalo de muestreo fuera de 0.1 s en lugar de 1 s?

El error de truncamiento de la fórmula de diferencias centrales es de orden $\mathcal{O}(h^2)$. Si se reduce el intervalo de $h = 1$ s a $h = 0,1$ s, el error teórico disminuye en un factor de 100 (ya que $(0,1)^2 = 0,01$, mientras que $(1)^2 = 1$). Esto mejoraría significativamente la precisión de la velocidad calculada. Sin embargo, en la práctica, reducir el intervalo de muestreo puede aumentar el ruido en las mediciones angulares, por lo que se debe encontrar un equilibrio entre la precisión numérica y la calidad de los datos.

4. Conclusión General

En esta evaluación se han aplicado exitosamente técnicas de interpolación numérica y diferenciación para resolver problemas de ingeniería:

- Los **splines cúbicos naturales** demostraron una excelente capacidad para aproximar respuestas transitorias de circuitos RLC, con errores por debajo de 10^{-2} .
- La **selección estratégica de puntos de muestreo** permitió alcanzar un MSE de 10^{-5} para el modelado de sistemas de segundo orden, superando el requerimiento de 10^{-3} .

- La ****diferenciación numérica por diferencias centrales**** permitió estimar velocidades a partir de datos angulares discretos, con errores de orden $\mathcal{O}(h^2)$.
- Se observó que los splines evitan el ****fenómeno de Runge**** que afecta a los polinomios de alto grado, siendo más robustos para aplicaciones prácticas.

FIN DE LA EVALUACIÓN